

416. Plan of Action

①

- ① why Transformers
- ② Architecture of Transformers
- ③ Self-Attention $\rightarrow Q, K, V$
- ④ positional Encoding
- ⑤ multi-Head Attention
- ⑥ Combining the working.

417. What and Why to use Transformers!

Transformers :- In NLP are type of deep learning model that use Self-Attention mechanisms to analyze and process natural lang. Data. They are Encoder and Decoder models that can be used for many applications, including machine Translation. $\rightarrow$ Specifically Sequence to Sequence Task

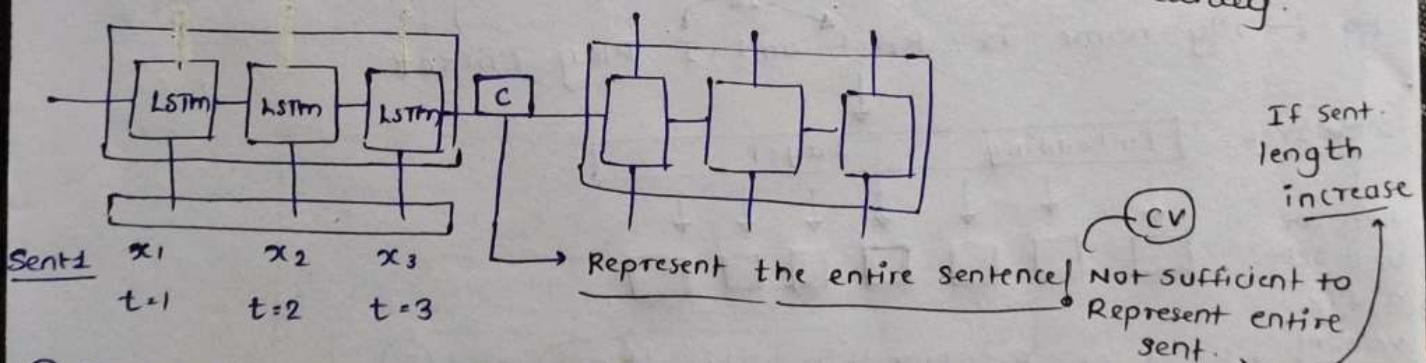
Eg :- Long. Translation $\rightarrow$ Google Translation ✓

English $\rightarrow$ French

iP many $\rightarrow$ oIP many

Length of sentence also meaningful Here!
Better for Accuracy.

Encoder - Decoder!



- ① Sentence 1 passing through Embedding layer it convert into the vector
- ② Then pass it through LSTM and after that we get/generate context vector :- Based on this we provide to next decoder layer!

$\therefore$ Bleu score is $\downarrow$ As Length of Sentence $\uparrow$

Problem With context vector we cannot send all at once so in this type of situation we actually take the help of attention mechanism

Problem with Attention mechanisms :-

① Parallely we cannot send all the words in the sentence

Dataset → Huge → Scable wr to Training

Transformers used → Self-Attention module

Scalable

Scalable

All the words will be parallely send! To encoder!

As we increases ↑↑ the data in Transformers we will get amazing State of Art models (SOTA) → specifically wr to NLP

• With the help of Transfer Learning $\xrightarrow{\text{used in}}$ Multi models $\rightarrow$ NLP + image Task

• Because of Transformers → AI space change → SOTA model (many)

BERT $\leftarrow$ GPT $\xrightarrow{\text{Help of}}$ Transfer Learning
(Train with Huge Data)

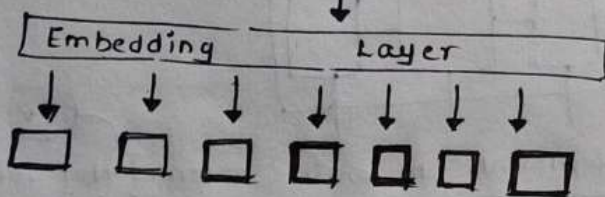
Related!

② Contextual Embedding :-

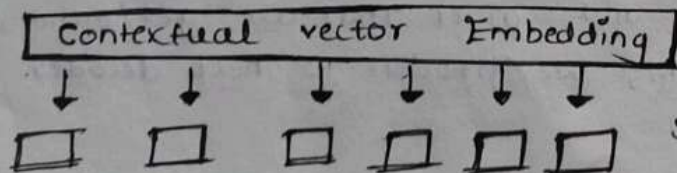
Eg :- My name is krish and I Play cricket.

Word2Vec

Here we will get a fix vector!



• Based on Relationship with (krish) and (I)

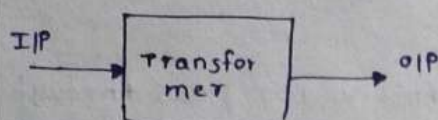


Should Have Relationship with other words!

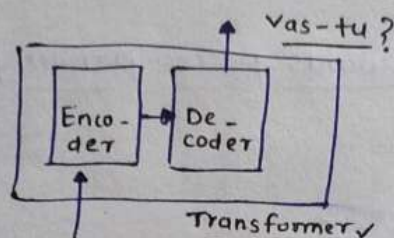
418. Basic Transformer Architecture.

3

Transformer used to solve
Sequence to Sequence Task!



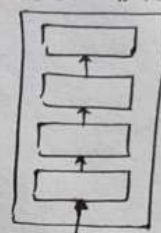
Eg :- Language Translation
(Eng To France)



How are
you?

Vas-tu?

- follow Encoder-Decoder Architecture
- Inside En-Decoder we have multiple like this

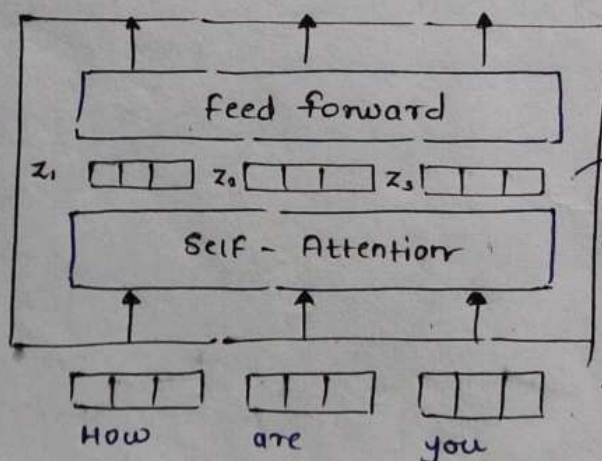
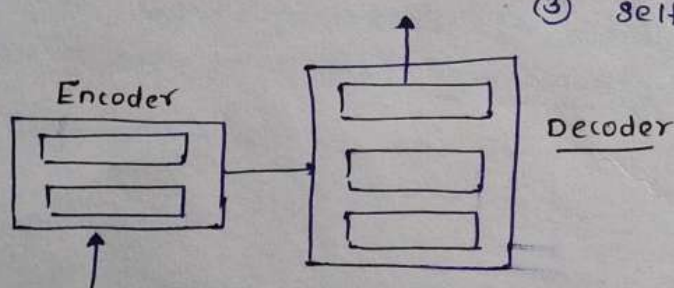


mostly in
Research paper
Used 6 Encoder
and Decoder!

• What is inside this Encoder?

→ In Single Encoder :- ① Feed Forward Neural NW
② self-Attention

→ In single Decoder :- ① Feed Forward NN
② Encoder-Decoder Attention
③ self-Attention.

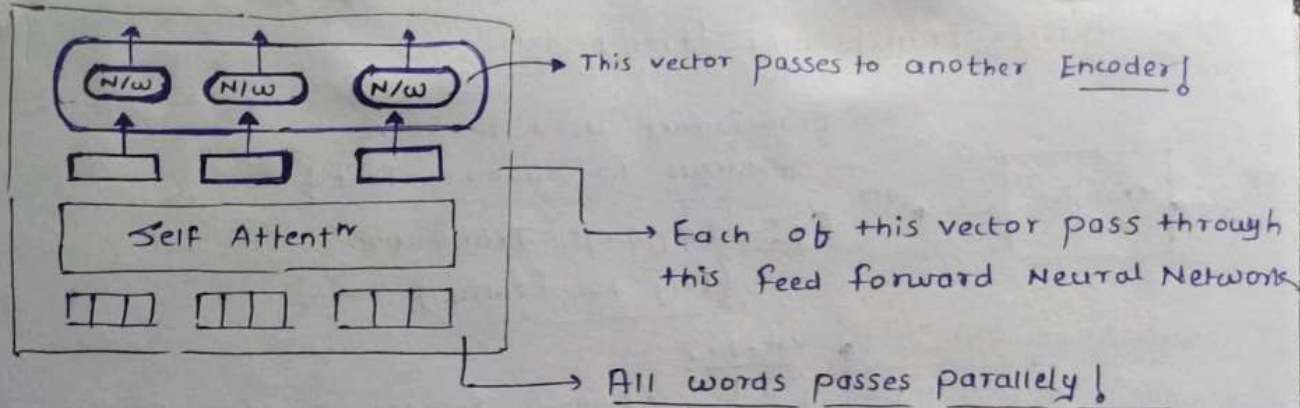


Contextual vector!

As p ss, this vector words
into Self-Attention and
it will convert this into
diff. vector! (z_1, z_2, z_3)

→ Converting all words into
vector

(Eg :- vector Emb.)



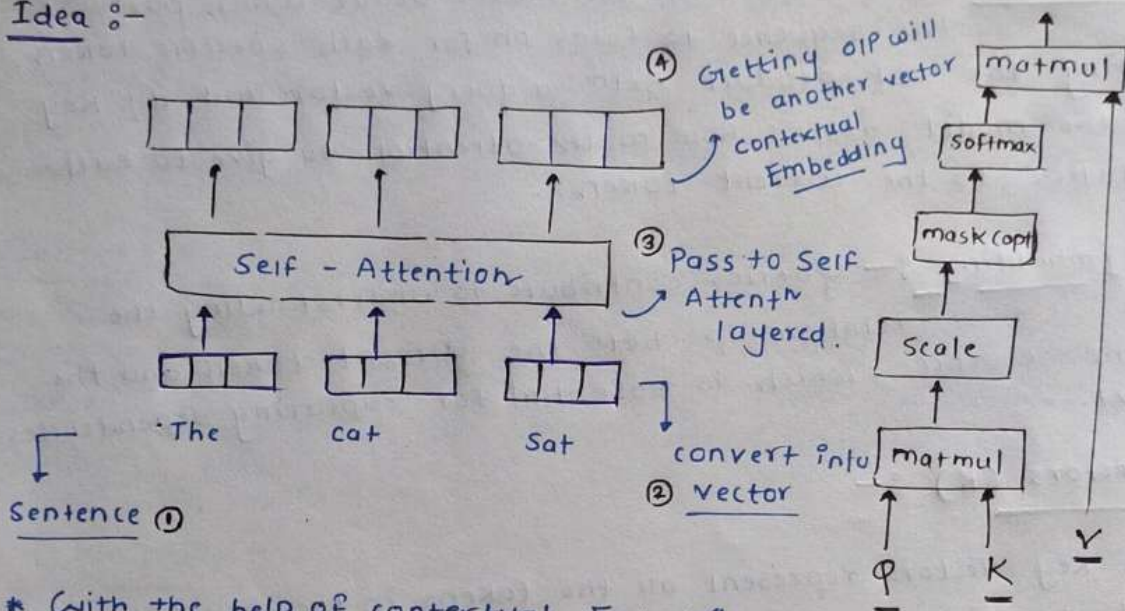
419. Self Attention Layer Working

(5)

Self Attention, also known as Scaled-Dot product attention, is a crucial mechanism in the transformer architecture that allows the model to weight the importance of different tokens in the i/p Sequence relative to each other.

Scaled-Dot attention

Idea :-



Sentence ①

* With the help of contextual Embedding will create that text Summarization, Lang. translation.

How Self-attention work on Scaled-Dot product attention.
 $Q \quad K \quad V \longrightarrow \text{vector.}$

① Embedding Layer? What actually Happend :-

we get fixed vector for each and every word

② Contextual vector? Based on context of other words

It actually Depend on entire sentence/dataset we actually have.

How we will able to convert Embedding vector into contextual Embedding.

① Create 3 imp. vectors i/p !

① Queries ② Keys ③ Values.

For every token if we convert it to Emb $\longrightarrow$ contextual vector
 we specifically create a model that will compute this vectors i/p.

① Query vectors (Q) :-

→ Query vectors represent the tokens for which we are calculating the attention. They help determine the importance of other tokens in the context of the current token.

Importance :-

Focus Determination :- Queries help the model decide which parts of the sequence to focus on for each specific token.

By calculating the dot product between a query vector and all key vectors, the model assesses how much attention to give to each token relative to the current token.

Contextual Embedding :- Queries contribute to understanding the relationship between the current token and the rest of the sequence, which is essential for capturing dependencies and context.

② Key vectors (K) :-

→ Role Key vectors represent all the tokens in the sequence and are used to compare with the query vectors to calculate attention scores.

Importance :-

Relevance Measurement :- Keys are compared with queries to measure the relevance or compatibility of each token with the current token. This comparison helps in determining how much attention each token should receive.

Information Retrieval :- Keys play a critical role in retrieving the most relevant information from the sequence by providing a basis for the attention mechanism to compute similarity scores.

③ Value Vector (v) :-

7

→ Value Vector hold the actual information that will be aggregated to form the oip of the attention mechanism.

Importance :-

Information Aggregation :- Values contain the data that will be weighted by the attention scores. The weighted sum of values form the oip of the self-attention mechanism, which is then passed on to the next layers in the network.

Context Preservation :- By weighting the values according to the attention scores, the model preserves and aggregates relevant context from the entire sequence, which is crucial for tasks like translation, summarization and more.

Example :-

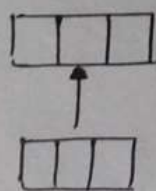
Input Sequence = ["The", "CAT", "SAT"]

Embedding size = 4 [means for every words will convert it into 4 vector]

Q, K and V → consider as 4 Dimension.

Step 1 :- Token Embedding

$$\left. \begin{aligned} E_{The} &= [1 \ 0 \ 1 \ 0] \\ E_{cat} &= [0 \ 1 \ 0 \ 1] \\ E_{sat} &= [1 \ 1 \ 1 \ 1] \end{aligned} \right\}$$
 so this the token that particularly convert for this words and this are my fixed vector.



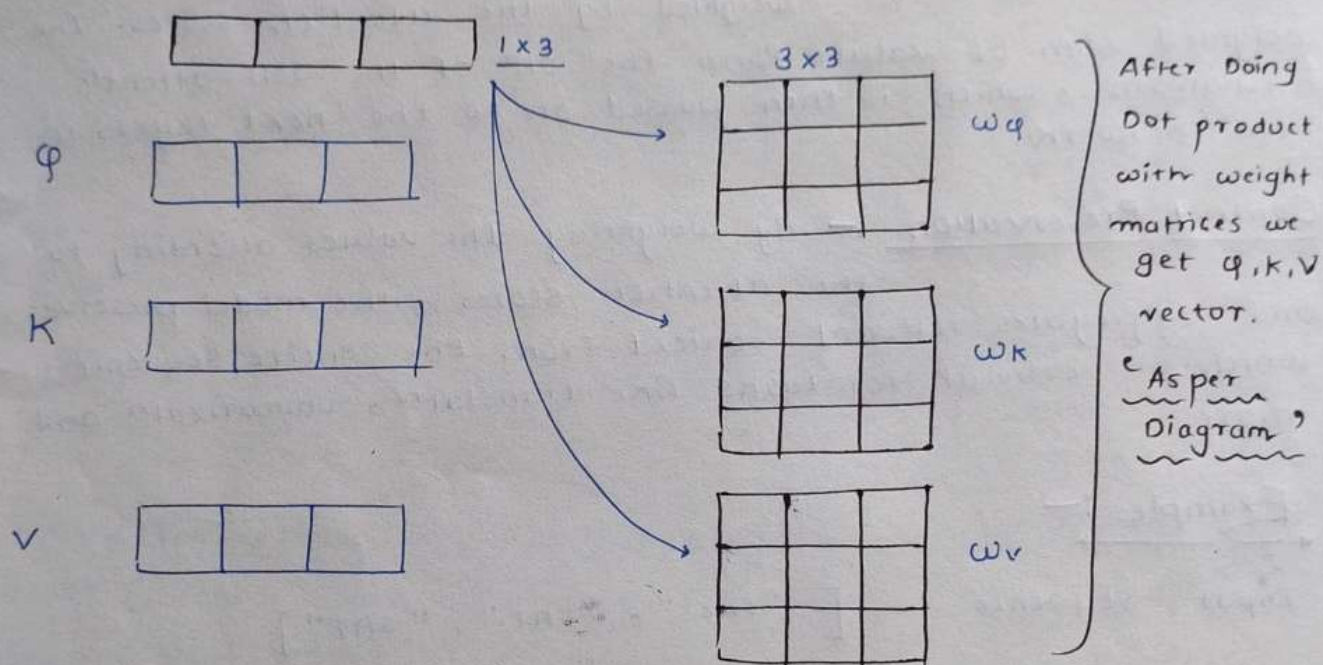
contextual Emb.

While converting sent. into cont. Embedding There's a Huge dependency for entire sentence.
for diff dataset we have diff. sentence.

Why we used this q, k, v vector Because,
The self attention mechanism have some information of other words/sent.
also is why the reason we used this.

② Linear Transformation :-

we will create q, k, v vector by multiplying the embedding by learn weights matrices ω_q, ω_k and ω_v .



$\omega_q = \omega_k = \omega_v = I \Rightarrow$ Identity matrix $\Rightarrow$ All Diagonal element 1 and other are zero.

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = I$$

When we do dot operation with identity matrix, we definitely get same value!

$$q_{The} = [1 \ 0 \ 10] \cdot I = [1 \ 0 \ 10] \checkmark$$

$$k_{The} = [\quad] \cdot I = [\quad] \checkmark$$

$$v_{sat} = [\quad] \cdot I = [\quad] \checkmark$$

Lets compute :- $q_{The} = k_{The} = v_{The} = [1 \ 0 \ 10]$

$$q_{cat} = k_{cat} = v_{cat} = [0 \ 0 \ 0 \ 1]$$

$$q_{sat} = k_{sat} = v_{sat} = [1 \ 1 \ 1 \ 1]$$

③ Compute Attention Scores :- Why?

9

The model assess how much attention to give to each token relative to the current token.

∴ For token the How much the important related to give cat and Sat.

Let's take the Dot operation.

The $\text{Score}(\varphi_{\text{The}}, K_{\text{The}}) = [1 \ 0 \ 1 \ 0] \cdot [1 \ 0 \ 1 \ 0]^T = 2$

$$\begin{matrix} \swarrow & \searrow \\ [1 \ 0 \ 1 \ 0] & \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} \end{matrix}$$

$$\text{Score}(\varphi_{\text{The}}, K_{\text{cat}}) \Rightarrow [1 \ 0 \ 1 \ 0] [0 \ 1 \ 0 \ 1]^T \Rightarrow 0$$

$$\text{Score}(\varphi_{\text{The}}, K_{\text{sat}}) \Rightarrow [1 \ 0 \ 1 \ 0] [1 \ 1 \ 1 \ 1]^T \Rightarrow 2$$

∴ This is how we compute score value!

Same for Token

SAT

For the Token **CAT**

$$\text{Score}(\varphi_{\text{cat}}, K_{\text{The}}) = [0 \ 1 \ 0 \ 1] \cdot [1 \ 0 \ 1 \ 0]^T \Rightarrow 0$$

$$\text{Score}(\varphi_{\text{cat}}, K_{\text{cat}}) = [0 \ 1 \ 0 \ 1] \cdot [0 \ 1 \ 0 \ 1]^T \Rightarrow 0$$

$$\text{Score}(\varphi_{\text{cat}}, K_{\text{sat}}) = [0 \ 1 \ 0 \ 1] \cdot [1 \ 1 \ 1 \ 1]^T \Rightarrow 2$$

$$[] []^T = 2 \checkmark$$

$$[] []^T = 2 \checkmark$$

$$[] []^T = 4 \checkmark$$

④ Scaling :-

↳ Means → Whatever Score we are getting!
why we performed the Scaling!

We take up the score and scale down by dividing the score by the sq. root of key vector.

$$\sqrt{d_k} \cdot d_k = 4 \rightarrow \sqrt{d_k} = 2$$



Dimension of Key vector. [Scaling in attention mechanisms is crucial to prevent the Dot product from growing too large

• In most Research paper → Ensure stable gradient during Training—

- If DK is Too large $\rightarrow$ Then ① Gradient Exploding
 $\rightarrow$ Dimension of key vector. Facing ② softmax Saturation
 2 prob.

For Example :- Consider 3 vector.

$$Q = [2 \ 3 \ 4 \ 1] \quad K_1 = [1 \ 0 \ 1 \ 0] \quad K_2 = [0 \ 1 \ 0 \ 1]$$

without scaling Let's do Dot product!

$$Q \cdot K_1^T = 2 \times 1 + 3 \times 0 + 4 \times 1 + 1 \times 0 = \boxed{6}$$

$$Q \cdot K_2^T = 2 \times 0 + 3 \times 1 + 4 \times 0 + 1 \times 1 = 0 + 3 + 0 + 1 = \boxed{4}$$

- Now after scaling we have to Applied Softmax Actⁿ function.
 To get final contextual Embedding.

* Score $[6, 4] \Rightarrow$ Scaling Not Applied.

Now softmax is applying for $[6, 4] =$

$$\left[\frac{e^6}{e^6 + e^4}, \frac{e^4}{e^6 + e^4} \right] \Rightarrow \left[\frac{e^6}{e^6(1 + e^{-2})}, \frac{e^4}{e^4(e^2 + 1)} \right]$$

$$\Rightarrow \left[\frac{1}{(1 + e^{-2})}, \frac{1}{(e^2 + 1)} \right] \Rightarrow [0.88, 0.12] **$$

- Most of the attention weight is assigned to the 1st key vector very little to the second vector,

One of the property of softmax in Deep Learning.

$\rightarrow$ Here the diff. Both score is very less But wr to OIP Here we see Huge diff

$$0.88 \rightarrow 0.6$$

$$0.12 \rightarrow 0.4$$

Similarly with $[10, 1] \rightarrow$ Here also we see Huge diff and value coming from Dot product of Q and K .

In the Scenerio like [High, less] value we actually do not apply Scaling. (11)

$$[10, 1] = [0.99, 0.01]$$

↓ | → If small value comes into picture when dot product coming as a large value!
Here we are facing problem of softmax saturation. / Also

* Now by Scaling this value!

$[0.99, 0.01]$ with scaling dot problem.

Known as Vanishing Gradient problem.

① Compute scaled dot product :-

score $[6, 4] \Rightarrow$ scale $\Rightarrow \sqrt{dk}$ (Divide by this) $\Rightarrow$ with $[6/2, 4/2] = [3, 2]$

$$\Downarrow$$
$$\sqrt{4} = 2$$

Now applying softmax Activation :- $([3, 2]) = \left[\frac{e^3}{e^3 + e^2}, \frac{e^2}{e^3 + e^2} \right]$

$$= \left[\frac{e^3}{e^3 [1 + e^{-1}]}, \frac{e^2}{e^2 (e^1 + 1)} \right]$$

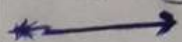
(Diff. is less/Normal)

$\Rightarrow [0.73, 0.27] \Rightarrow$ Also known as Attention Weights

• Here the attention weight are more balanced compared to the unscaled case :-

* Summary Importance :-

- ① Scaling prevent extremely large dot products,
- ② Actually help in stablizing the gradients during backpropagation.
- ③ Making training process more stable and efficient.



① Prevent saturation By scaling dot products, the softmax function produces more balanced attention weights, preventing the model from focusing too heavily on a single token and ignoring others.

② Improved Learning and Balanced attention weights enable the model to learn better representations by considering multiple relevant tokens in the sequence, leading to better performance on tasks that require context understanding.

Why we specifically divide by $\sqrt{d_k}$ $\Rightarrow$ Variance (Whenever Dot product Do!)
(As we keep _____ on increasing the Dimension the Dot product Dimension $\uparrow$ variance $\uparrow$)
 $\sqrt{d_k}$ (Variance for that particular Dot product is almost same)

* Scaling ensures the Dot product are kept within a range that allows the softmax function to operate effectively providing a more balanced distribution of attention weights and improving the overall learning process of the model.

Now we have to Apply scaling :-

$$\sqrt{d_k} = \sqrt{4} = 2$$

$$\begin{aligned} \text{Scaled-Score} (Q_{The}, K_{The}) &= 2/2 = 1 \\ \text{Scaled-Score} (Q_{The}, K_{car}) &= 0/2 = 0 \\ \text{Scaled-Score} (Q_{The}, K_{sat}) &= 2/2 = 1 \end{aligned} \left. \vphantom{\begin{aligned} \text{Scaled-Score} (Q_{The}, K_{The}) &= 2/2 = 1 \\ \text{Scaled-Score} (Q_{The}, K_{car}) &= 0/2 = 0 \\ \text{Scaled-Score} (Q_{The}, K_{sat}) &= 2/2 = 1 \end{aligned}} \right\} \begin{array}{l} \text{Similarly Scaling will be} \\ \text{Done for all other tokens.} \end{array}$$

⑤ Apply Softmax

Whenever we find out we Basically apply and create Attention weight.

$$\text{Attention weights 'The'} = \text{Softmax}([1, 0, 1]) = [\underline{0.423}, \underline{0.1554}, \underline{0.4223}]$$

$$\text{Attention weights 'car'} = \text{Softmax}([0, 2, 2]) = [0.1554, 0.4222, 0.4223]$$

$$\text{Attention weights 'SAT'} = \text{Softmax}([2, 2, 4]) = [0.2119, 0.2119, 0.5762]$$

⑥ Weighted Sum of values :-

we multiply attention weights by corresponding values vectors for the token the =

$$\text{Output}_{(\text{The})} = 0.4223 * V_{\text{The}} + 0.1554 * V_{\text{cat}} + 0.4223 * V_{\text{sat}}$$

Now we have to take the Token Embedding (3E)

$$= 0.4223 * [1 \ 0 \ 1 \ 0] + 0.1554 * [0 \ 1 \ 0 \ 1] + 0.4223 * [1 \ 1 \ 1 \ 1]$$

$$= [0.4223, 0, 0.4223, 0] + [0, 0.1554, 0, 0.1554] + [0.4223, 0.4223, 0.4223, 0.4223]$$

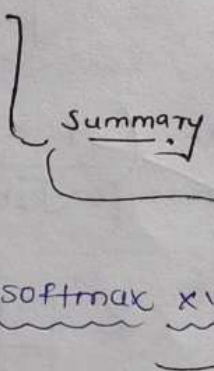
$$= [1.2669, 0.9999, 1.2669, 0.9999]$$

So the Embedding The keyword

1	0	1	0
---	---	---	---

 → Self Attention

- ① $\Phi \ K \ V \quad \omega^Q, \omega^K, \omega^V$
- ② Attention Score
- ③ Scaled
- ④ ~~Attention~~ Softmax
- ⑤ weighted Sum of value (softmax x v)

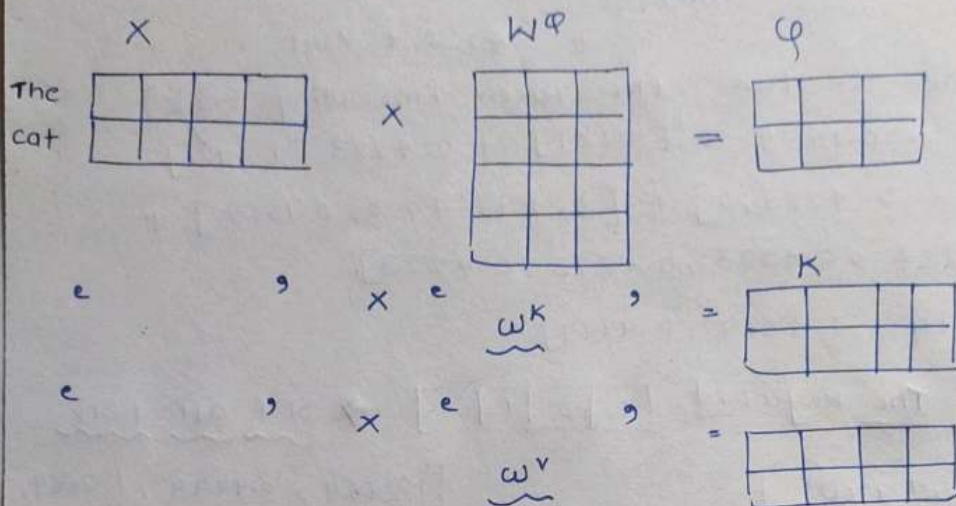


$[1.2669, 0.9999, 1.2669, 0.9999]$
Contextual vector

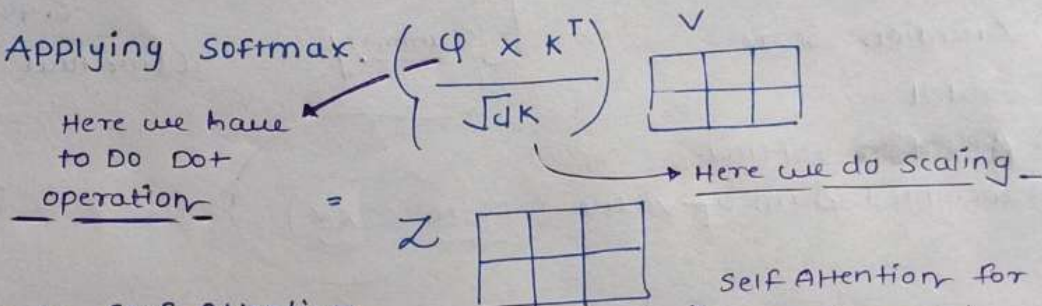
420. Multi-Head Attention

<http://jalammar.github.io/illustrated-transformer/>

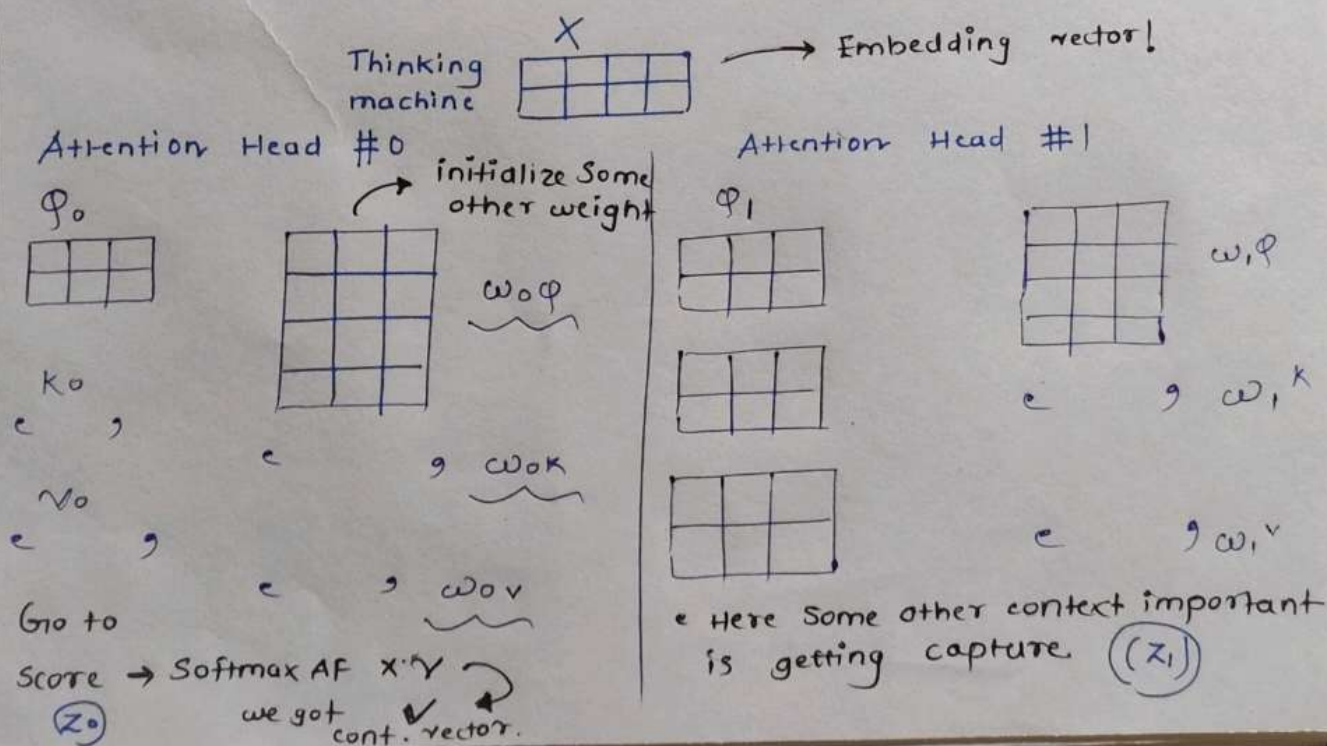
Visualize
Transformer
Understanding



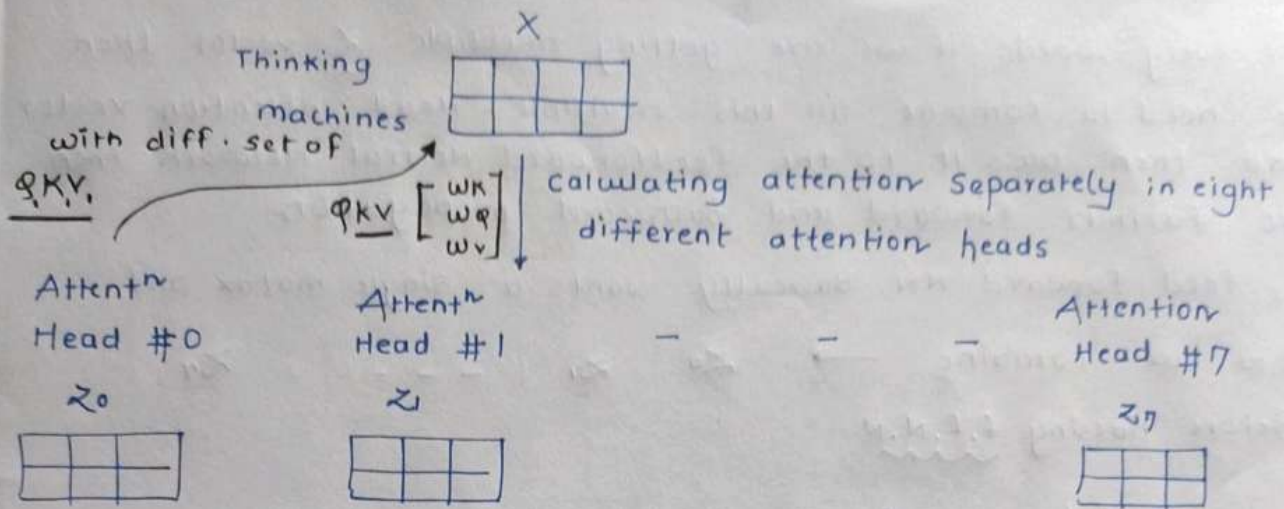
Then Now Applying softmax.



Now we have to Self-Attention with multi-head Attention.



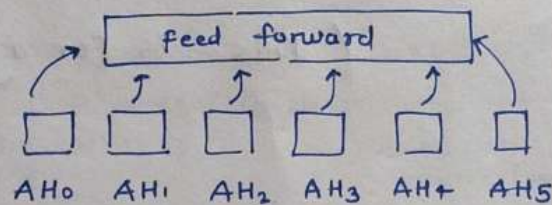
- It expands model's ability to focus on different positions of tokens. (15)



- # With a diff. set of ϕ_{KV} we get diff. z_0 .
- # Here instead of one Attention Head we calculate diff. diff. multiple Head (z_0)

Here we focus on multiple Head attention

Basically



4.2.1. Feed forward Neural Netⁿ with MHA

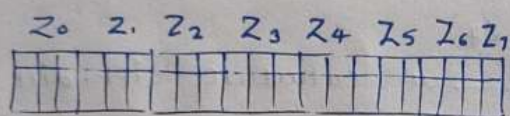
For every words if we are getting multiple Z vector then we need to combine all this multiple - Head attention vector and then pass it to the feedforward Neural Networks then Do further forward and Backward propagation.

So, feed forward NN Basically wants a single matrix and

How we combine $\rightarrow Z_0 \quad Z_1 \quad \dots \quad Z_7$

Before passing f.f.N.N.

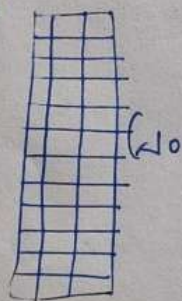
- ① we need to concatenate all the attention Heads



(7 multiple Head Attention)

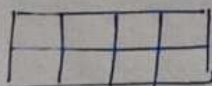
- ② Multiply with a weight matrix w^* that was trained jointly with the model

$\times$
(multiply)



(This is for a Single Words)

- ③ The result would be the Z matrix that captures information from all the attention Heads. We can ~~the~~ send this forward to the ffNN.



- ④ Then we have to calculate our ' Z '

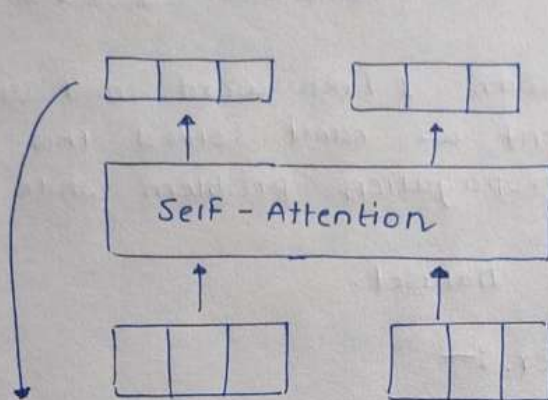
So whatever ' Z ' will get and will pass it to next encoder

• Highly co-related $\rightarrow$ means Having High Dependency in multiple Head Attention.

422. Positional Encoding Indepth Intuition

(17)

* Representing - Order of Sequence



Advantage :-

① Words tokens process parallelly

Drawbacks (Becoz of this)

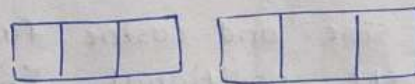
It lacks the Sequential Structure of the words (The orders Basically)

Just Because of the order of Sentence is change the meaning of sentence is change. All words are cons. Here,

suppose if we pass it to Self-Attention then Both this words will return with this same vector

for Eg :-

- ① Lions Kills Tiger
- ② Tiger Kills Lions



Actually positional Encoding vector.

Learnt? Because of ordering of Sent. the entire sentence/meaning is change. So this information is missing in the Transformer that is why we use positional Enc to move forward in our problem Statement.

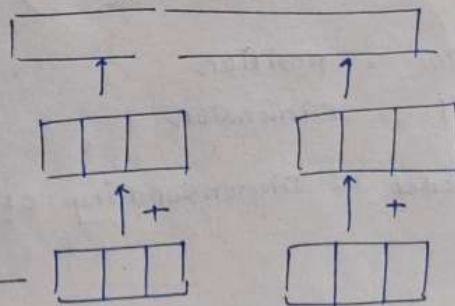
• Positional Encoded will be added to normal vector

→ Responsible for actually telling which words are in which placed Basically.

* According To Research paper

Will be added to normal vector

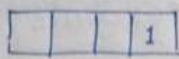
Positional Encoding



There are multiple ways ① Embedding Ways.

Let's take 4 Dimension

(Actually Represent position)



* Suppose if we have Book with 1 Lakh words and usually this type of Problem statement we can't solve this because it generally gives Back propagation problem while updating weight.

→ While considered Huge Dataset.

According To Research paper :-

Adding P.E.V which is same size of Embedding vector itself.

Type of Positional Encoding

① Sinusoidal P.E

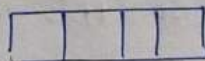
② Learned P.E



It uses sine and cosine functions of different frequencies to create positional Encodings.

→ With the help of P.E will not have unlimited because S.P.E all the values will be ranging between -1 to 1

How it works :-



$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

All the values will be replaced by this formula :-

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Where , pos = position
i = Dimension.

d_{model} = Dimensionality of embedding

Eg :- The cat sat

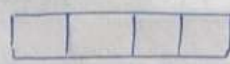
converting into vector.

(19)

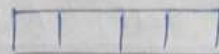
The $\rightarrow [0.1 \ 0.2 \ 0.3 \ 0.4]$

CAT $\rightarrow [0.5 \ 0.6 \ 0.7 \ 0.8]$

SAT $\rightarrow [0.9 \ 1.0 \ 1.1 \ 1.2]$



E.V



P.E

4 dimension

Embedding vector

$$PE_{(pos, 2i)} = \sin\left(\frac{Pos}{10000^{2i/d_{model}}}\right) \quad \left| \quad PE_{(pos, 2i+1)} = \cos\left(\frac{Pos}{10000^{2i/d_{model}}}\right)$$

* Now we have to generate 4 positional Enc. vector!

For our Example $d_{model} = 4$

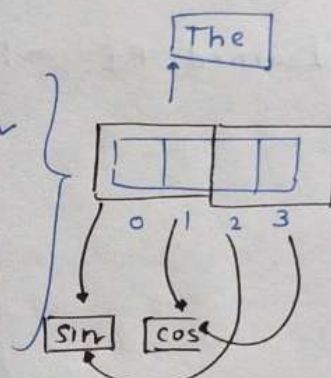
For position $pos = 0$

$$P.E(0, 0) = \sin\left(\frac{0}{10000^{0/4}}\right) = \sin(0) = 0 \checkmark$$

$$P.E(0, 1) = \cos\left(\frac{0}{-}\right) = \cos(0) = 1 \checkmark$$

$$P.E(0, 2) = \sin\left(\frac{0}{10000^{2/4}}\right) = \sin(0) = 0 \checkmark$$

$$P.E(0, 3) = \cos(0) = 1 \checkmark$$



why like this?

Not getting similar

like of Encoded value

(With respect to first words the
positional Encoding $[0 \ 1 \ 0 \ 1]$)

Now for $P.E = 1 \rightarrow$ CAT

$$P.E(1, 0) = \sin\left(\frac{1}{10000^{0/4}}\right) = \sin(1) = 0.8415$$

$$P.E(1, 1) = \cos\left(\frac{1}{10000^{2 \times 1/4}}\right) \approx 0.5403$$

$$P.E(1, 2) = \sin\left(\frac{1}{10000^{2/4}}\right) \approx 0.01$$

$$P.E(1, 3) = \cos(-) \approx 0.99995$$

P.E

The $[0.1 \ 0.2 \ 0.3 \ 0.4]$

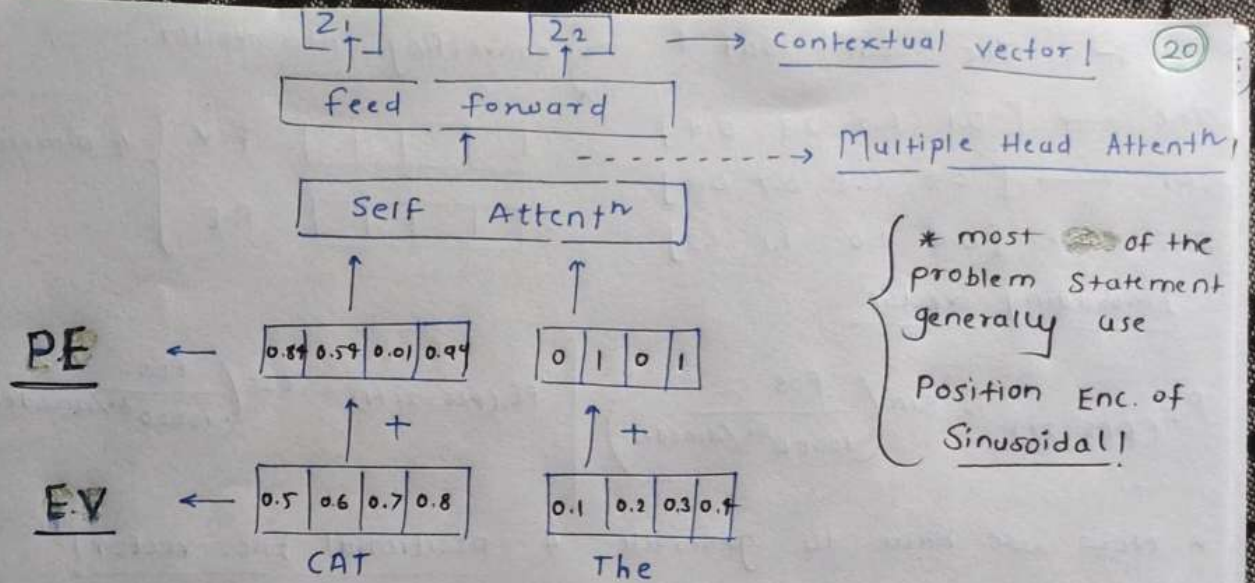
CAT $[0.5 \ 0.6 \ 0.7 \ 0.8]$

P.E for this
vector

$[0 \ 1 \ 0 \ 1]$

$[0.84 \ 0.54 \ 0.01 \ 0.99]$

if we get similar
Kind of Encoded value
then will miss order
of elements.



(2) Learned P.E : ————— P.E are learned During training

423) Layer Normalization

(21)

* **Residual** :- Whatever vector we get before adding the positional encoding to the i/p embedding that is also passed to the add and normalized.

Provide some additional signal to the layer normalization!

column ✓

* Where this addition and normalization is added?

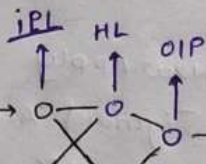
* Talking about the steps?

- ① This is our i/p sentence
- ② we embed each words → Here we add positional Encoding there
- ③ Split into 8 heads
- ④ Calculate the attention using the resulting Q/K/V matrices
- ⑤ Concatenate the result & matrices then ~~matrix~~ multiply weight matrices W^o to produce the o/p of the layer

This is to add and Normalize there will do e Layer Normalisation.
→ we are providing signal to Entire layer and with some additional signal.

Normalization :-

- Batch Normalize F_1/x_1
- Layer Normalize F_2/x_2



Dataset :-

House Size	Area No. of Rooms	Price
F_1	F_2	
1200	2	45
1500	3	70
2000	3.5	80

• Before providing all i/p we do normalization

→ Standard Scaling → we take each of this feature and Apply general Standard Scaling! formula

$$Z_{score} = \frac{x_i - \mu}{\sigma}$$

$x_i \rightarrow$ i/p

$\mu \rightarrow$ mean

$\sigma \rightarrow$ Standard Deviation

we apply such a way $\mu=0$ and $\sigma=1$

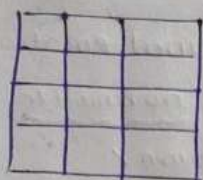
Talking F_1 and After Apply S.S will get $\rightarrow F_1'$

$\mu=0$ and $\sigma=1$

∴ Basically applied to entire column!

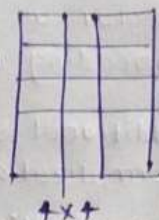
So we get H, σ then we can apply Standard Deviation!

Deep learning :- I/P Image $\longrightarrow$ min max scaler! (22)



After Applying
the maxmin
Scaler

we have all
Dimension into
0-1



out of Range

Have some images
of 4x4 matrix
value is betn
0-255.

• Advantage :- ① Improved Training Stability

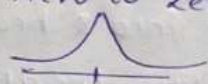
• Suppose if Back Propagation is
Happend! then will not get
any type of vanishing gradient
& Exploding Grad problems.

② Due to this upper process
 $\longrightarrow$ will get ~~fast~~ faster
convergence!

(Here we have multiple
Distributⁿ

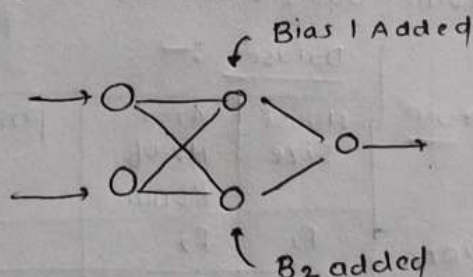
So we convert this distributⁿ
where all my informatⁿ is
centered! zero to zero

After
Apply
Scaling



Zero Centered!

In Back Propagation $\Rightarrow$ It is stably update



Now Apply
Sigmoid Actⁿ
functⁿ

$$Z_1 = \sigma(0.45 * w_1 + 0.55 * w_3) + b_1$$

$=$ value 1

$$Z_2 = \sigma(\quad) + b_2$$

$=$ value 2

House Size	Room	price
0.45	0.55	45
0.60	0.20	-
-	-	-

Now the value 1 and
value 2 distribution
is actually changed wr to
f1 and f2

We dont need that change in distribution
then we have to perform normalization
in every o/p Z_1 and Z_2 $\sim$ Batch Normalizer

Apply Kma Pdenga formula $\mu, \sigma_1, \mu_2, \sigma_2$

* Batch Normalization

① f_1, f_2
Taking comp
 z_1

z_1	z_2

②

(v/s)

Layer Normalization (In Transformers)

① f_1, f_2, z_1, z_2

② We are taking
Here layer by layer

(23)

* Gamma and Beta! (γ, β) → Learning Parameters

Suppose

• IN ANN
Batch Normalization

z_1	z_2
-	-
-	-
-	-
0	-
0	0
0	0

After applying
normalization
layer/layer
Because every words
is important
• Also need to normalize
zero value!

In Batch normalization
There is a little change in
No.
• Talking layer by layer

• Is this compulsory that we need to do normalization?
→ What if, final o/p and distribution is important for predicting o/p!

• If we do not want to normalization this!

→ we using γ, β $z_1 = \sigma[w_1^T x + b_1]$

Assign
Learnable
parameter!

$$y = \gamma \left[\frac{z_1 - \mu_1}{\sigma} \right] + \beta$$

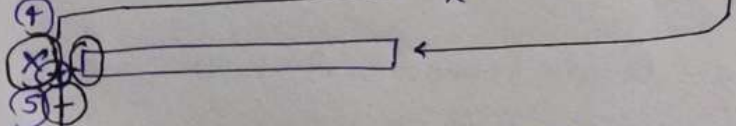
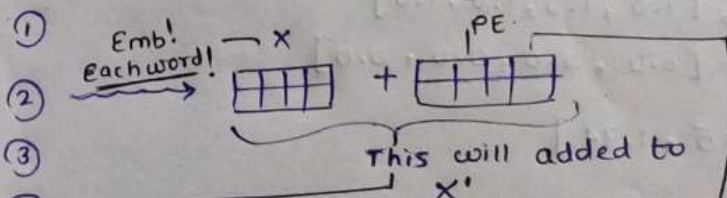
• $z_1 =$ all no.

Learnable
parameters

Learned Scale ✓

γ and β → Shift

all called as 'Scale & Shift parameters'



Here we specifically Apply layer normalization!

How this vector is normalized?

4 Dimension

Eg :- 'CAT' = $[2.0, 4.0, 6.0, 8.0]$ → ①

Talking
Example
As Token

parameters = $\gamma = [1.0, 1.0, 1.0, 1.0]$ → ②

$\beta = [0.0, 0.0, 0.0, 0.0]$ →

424 Layer Normalization Examples :-

(24)

(i) Compute mean $\rightarrow Z_{\text{score}} = \frac{x_i - H}{\sigma}$ $\left\{ \begin{array}{l} H = \frac{1}{4} (2.0 + 4.0 + 6.0 + 8.0) \\ = \frac{20.0}{4} = \underline{5.0} \end{array} \right\}$

(ii) Compute the variance (σ^2)

$$\sigma^2 = \frac{1}{4} [(2.0 - 5.0)^2 + (4.0 - 5.0)^2 + (6.0 - 5.0)^2 + (8.0 - 5.0)^2]$$

$$= \underline{5.0}$$

(iii) Normalize the IP

$$\hat{x}_i = \frac{x_i - H}{\sqrt{\sigma^2 + \epsilon}}$$

$\because \epsilon = 1e^{-5} \rightarrow$ Basically for Avoid Divide by zero!

$$\sqrt{\sigma^2 + \epsilon} = \sqrt{5.0 + 1e^{-5}}$$

$$\approx \sqrt{5.000} = \underline{2.236}$$

$\because$ Now normalize Each and Every w.r to Numbers!

$$\hat{x}_1 = \frac{2.0 - 5.0}{2.236} \approx \underline{-1.34}$$

$$\hat{x}_2 = \frac{4.0 - 5.0}{2.236} \approx \underline{-0.45}$$

$$\hat{x}_3 = \frac{6.0 - 5.0}{2.236} \approx \underline{0.45}$$

$$\hat{x}_4 = \frac{8.0 - 5.0}{2.236} \approx \underline{1.34}$$

Basically Normalized vector $\Rightarrow \underline{\hat{X} = [1.34, -0.45, 0.45, 1.34]}$

(iv) Scale and Shift :-

$$y_i = \gamma_i \hat{x}_i + \beta_i$$

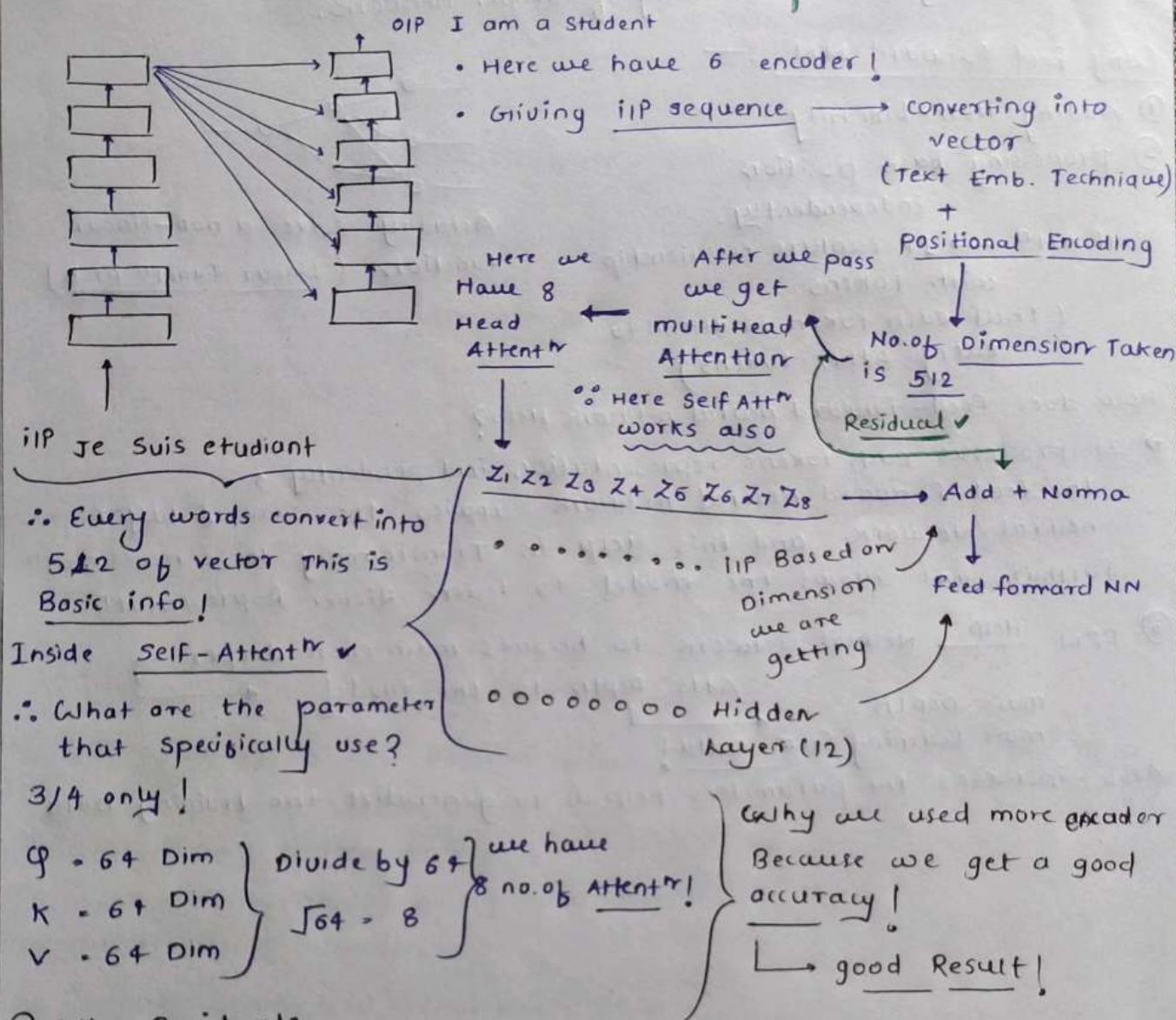
$$\gamma = [1.0, 1.0, 1.0, 1.0]$$

$$\beta = [0.0, 0.0, 0.0, 0.0]$$

$$y = [-1.34, -0.45, 0.45, 1.34]$$

425. Complete Encoder Transformer Architecture

25



① Why Residuals

②

③ Why Used feedforward neural network!

Residuals :- Also Known as 'Skip connection' why it Required?

① Addressing vanishing gradient problem. As no. of layers increases that Gradient Descent with resp to loss function is very small.

How Residuals Help :- Residual connection create a short paths for gradients to flow directly through the n/w Becoz of this gradient Remain sufficiently Sharp. large.

- ② It improved Gradient flows :- Convergence will be faster then Trainer is also Smoother!
- ③ This actually enables training of deeper networks.

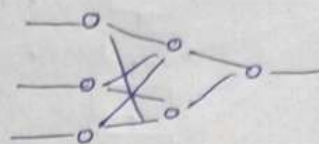
26

Why feed forward NN :-

- ① Adding Non-Linearity
- ② processing Each position independently.

Self Attn → It capture relationship with tokens

(That each token is attend to every other tokens)



Actually solve a non-linear function. (Linear function prob)

How does feed-forward neural network help?

- It processes each tokens representation independently ,
The feed-forward neural network applies the same 2 layer neural network and this help in Transforming these representatr furthur and allows the model to learn Richer Representatr.

- ③ FFN Help → Neural Network to becomes much more deeper
- Adds Depth to the model
- more Depth
- more Learning from Data!

Also increases the parameters help us to generalize the training Data.

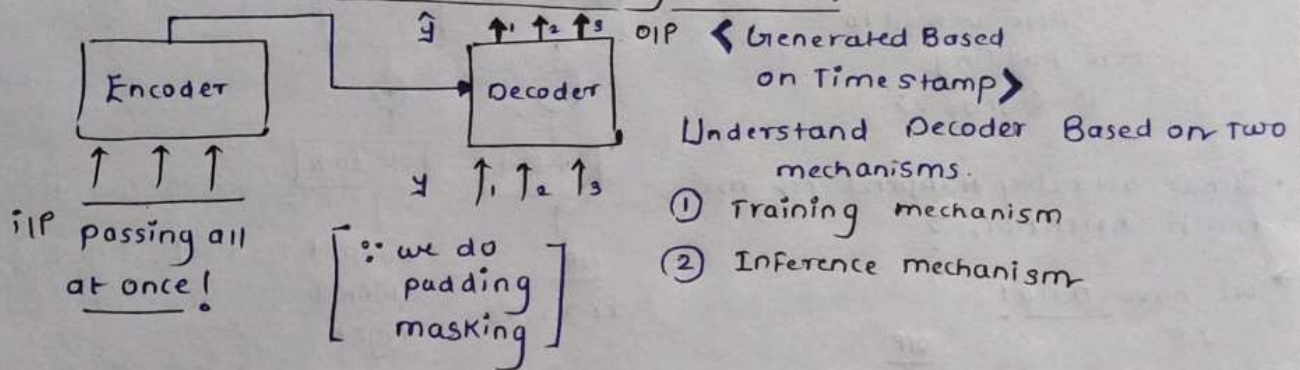
426. Decoder in Transformers

(27)

- 3 main components →
- ① masked multi-Head Attn
 - ② multi-Head Attn (Enc-Decod. Attn)
 - ③ feed forward neural net.

The Transformer decoder is responsible for generating the O/P sequence one token at a time, using the encoder's output and the previously generated tokens.

• How Encoder and Decoder actually works.!



① Multi-masked-Head Attention.

- ① I/P Embedding and Positional Embedding → Here we zero padding to make our sequences length Equal!
- ② Linear projection for QKV
- ③ Scaled Dot projection Attnⁿ
- ④ Mask Application. } → Try to understand the imp.
- ⑤ multi-Head Attention
- ⑥ Concatenation and Final Linear program
- ⑦ Residual connection and Layer Normalization.

- ②
- ③
- ④
- ⑤

427. Decoder Transformer - Masked Multi-Head Attention Working

28

Dataset :-

English $\rightarrow$ Hindi

$\langle x_1, x_2, x_3 \rangle$
(3 words)
wr. Eng.

$\langle y_1, y_2 \rangle$
(2 words)
wr. Hindi.

consider one
i/p Data!

Here we need to
zero padding
ie $\langle y_1, y_2, 0 \rangle$

• What actually happen in mask
multi Attention?

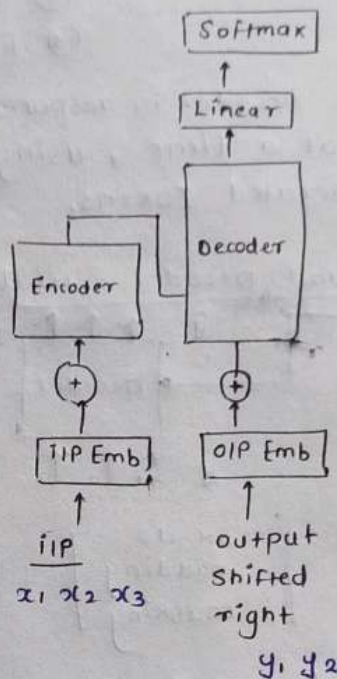
$\rightarrow$ We have Data!

I/P

O/P

[4 5 6 7]

[1 2 3]



① First Step in Decoder is (i) I/P Embedding and positional Encoding
O/P consider as padding and we use 1 O/P Here! $\rightsquigarrow$ 0 padding.
Now output Embedding shifted to Right :-

[0.1, 0.2, 0.3, 0.4]

[0.5, 0.6, 0.7, 0.8]

[0.9, 1.0, 1.1, 1.2]

[0.0, 0.0, 0.0, 0.0]

• Now the output Embedding should
Specifically be going along with
positional Encoding.

PE = 0

$\rightarrow$ Now we further go to masked multi Head Attention.

There are few step :- ① Linear projection of K V

② Scaled Dot product - calc. Attn Score

③ Masked Application $\rightarrow$ Looked Ahead
 $\rightarrow$ Padding

Second step :- Linear Projection for QKV

Creating query (Q), Key (K) and value (V) vectors.

$\omega_Q = \omega_K = \omega_V = I \rightarrow$ considered as Identity matrix

$\Phi = \text{output Embedding} + w\Phi = \text{output Embedding}$

(29)

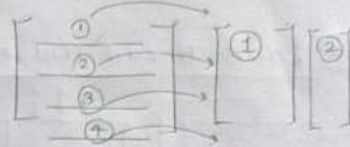
$K = e \dots + wK = OE$

Because of identity matrix!

$V = e \dots + wV = OE$

We will get $\Phi = K = V$ $\begin{bmatrix} 0.1 & 0.2 & \dots \\ \vdots & \vdots & \vdots \\ 0.0 & 0.0 & 0.0 \end{bmatrix}$

$\Phi = K = V =$



③ Scaled Dot Product Attention calculation!

How do our

Scores = $\Phi * K^T / \sqrt{d_K} \rightarrow \Phi * K^T / 2$

Now calculating score :-

What is Transpose of K^T (vertical matrix Basically)

After calculating my final score operator

$\begin{bmatrix} 0.3 & 0.7 & 1.1 & 0.0 \\ 0.7 & 1.9 & 3.1 & 0.0 \\ 1.1 & 3.1 & 5.1 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 \end{bmatrix}$

$\begin{bmatrix} 1 \times 1 + 1 \times 2 + \\ 1 \times 4 + 1 \times 5 + 2 \times 1 + 2 \times 2 + \\ 3 \times 1 + 3 \times 2 + 4 \times 1 + 4 \times 2 \\ \text{(NO Sir Rule)} \end{bmatrix}$

④ Masked Application :-

It helps manage the structure of the sequences being processed and ensures the models behaves correctly during training and Inferencing.

purpose :-

- ① To Handle Sequences of different length in Batch.
- ② To ensure that padding tokens, which are added to makes sequences of uniform length, Do not affect the model prediction.

Eg :- sequence1 [1, 2, 3]

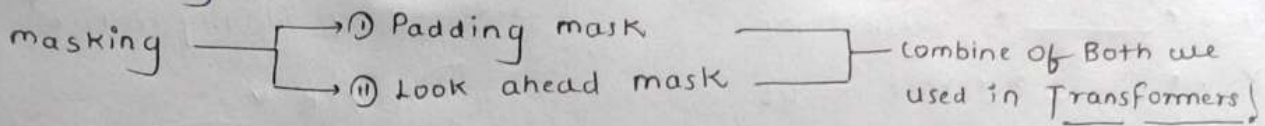
To encoder / Sequence 2 [4, 5, 0] $\rightarrow$ 0 is the padding Token

To Decoder send

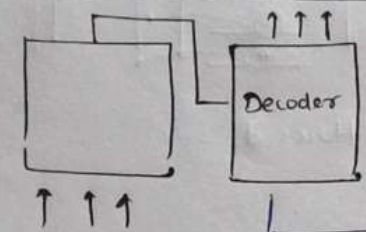
$\uparrow$ Influence the Attention mechanisms

$\downarrow$ Lead to Incorrect or Biased Prediction.

A padding mask → The Token are ignored



② Look Ahead mask :- Used To maintain Auto Regressive property



Whenever we passed in any i/p we actually passed Decoder 1 by 1

How are you

Agar we want next words then we must some information of previous words / OP.

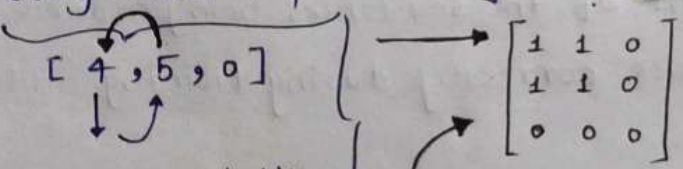
Q) why looked Ahead mask is specifically used ?

Q) Why do we maintaining autoregressive property to ensure that each each position.

① To ensure that each position in Decoder o/p ~~sequence~~ sequence can only attend to the previous position, but no future posⁿ.

② for Diff. Seq-Seq. Task :- language modeling, Translation.

Eg :- [4, 5, 0] → [1, 1, 0] ^{Zero padded Token.} Convert 1D To 3D.
we can also extended to 3 Dimension.



1D → 2D

In order to convert 1D to 2D,

For each Token in the Sequences, the mask should indicate which

Token is attend to!

Token 1 attends to token 1, 2
1, 2

④ should probably Take the context of Token ⑤ Similarly with ⑤ ④

Not Taking the context of zero!

Attention mechanisms

* Now calculating look ahead mask :-

(31)

→ Depends on Decoder o/p

→ $\begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$ • Now we combine and looking Ahead mask

$\begin{bmatrix} 1 & 1 & 0 \end{bmatrix}$ • How?

$\begin{bmatrix} 1 & 1 & 1 \end{bmatrix}$ → Basically with help of two mask with element wise!

Combine Mask → $\begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$ } wherever the value is value
 $\begin{bmatrix} 1 & 1 & 0 \end{bmatrix}$ } in combine mask!
 $\begin{bmatrix} 1 & 1 & 1 \end{bmatrix}$ } there are specify infinity

Compute our mask :-

Scores :- $\begin{bmatrix} \end{bmatrix}$ Look Ahead mask $\begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix}$
 $\begin{bmatrix} \end{bmatrix}$ $\begin{bmatrix} 1 & 1 & 0 & 0 \end{bmatrix}$
 $\begin{bmatrix} \end{bmatrix}$ $\begin{bmatrix} 1 & 1 & 1 & 0 \end{bmatrix}$
 $\begin{bmatrix} \end{bmatrix}$ $\begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix}$

* If i want to use padding masking [Extended to 2D format]

$\begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$

* If we want to probably find out combined mask →

Look Ahead * Padding Mask

$\begin{bmatrix} 1 \times 1 & 0 \times 1 & 0 \times 1 & 0 \times 0 \\ 1 \times 1 & 1 \times 1 & 0 \times 1 & 0 \times 0 \\ 1 \times 1 & 1 \times 1 & 1 \times 1 & 0 \times 0 \\ 1 \times 1 & 1 \times 1 & 1 \times 1 & 1 \times 1 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & -\infty & -\infty & -\infty \\ 1 & 1 & -\infty & -\infty \\ 1 & 1 & 1 & -\infty \\ 1 & 1 & 1 & -\infty \end{bmatrix}$

* Compute masked Score → wherever we have zero

$\begin{bmatrix} -0.3 & -\infty & -\infty & -\infty \\ 0.7 & 1.9 & -\infty & -\infty \\ 1.1 & 3.1 & 5.1 & -\infty \\ 0.0 & 0.0 & 0.0 & -\infty \end{bmatrix}$

$-\infty$ is added to zero value!
 It is Basically zero out/
 Removed the influence when the
 Softmax Actⁿ functⁿ is applied!

* Softmax

Score = Softmax (masked Scores)

$$\text{Softmax} = \begin{bmatrix} [1.0, 0.0, 0.0, 0.0] \\ [0.3, 0.7, 0.0, 0.0] \\ [0.1, 0.3, 0.6, 0.0] \\ [1.0, 0.0, 0.0, 0.0] \end{bmatrix}$$

* Weighted Sum of value

$$\text{Attention OIP} = \text{Softmax Score} * V$$

* Masking → masking in the transformer architecture is essential for several reasons. It helps manage the structure of the sequences being processed and ensures the model behaves correctly during training and inference. Here are the key reasons for using masking:

① Handling variable-length sequence with padding mask

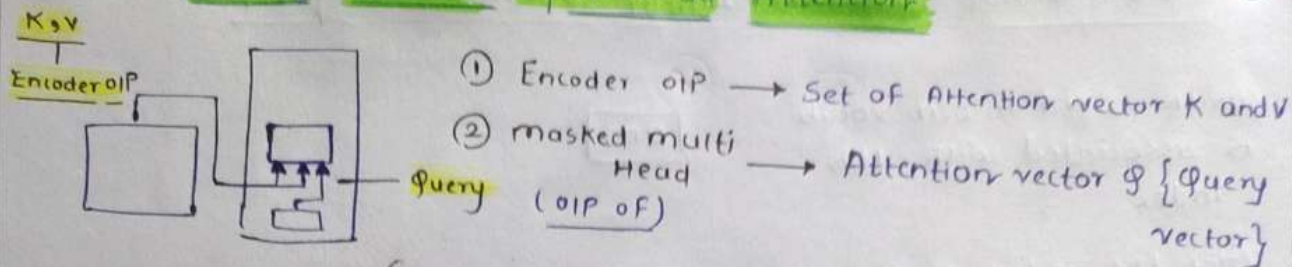
purpose → To handle sequences of different length in a batch. To ensure that padding tokens, which are added to make sequences of uniform length, do not affect the model's prediction.

② Maintaining Autoregressive property with look-ahead mask

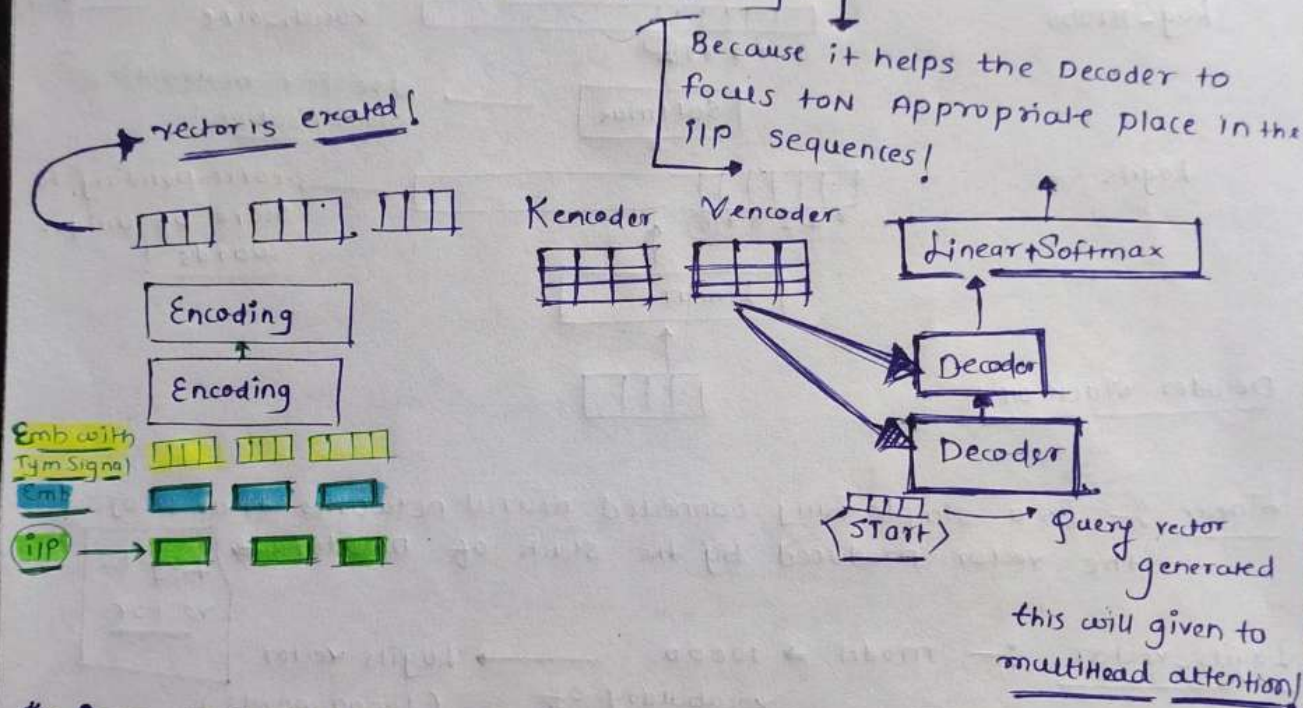
purpose → To ensure that each position in the decoder OIP seq. can only attend to previous positions and itself, but not future positions. This is crucial for sequence generation tasks like language modeling and translation, where the model should not have access to future tokens when predicting the current token.

428. Encoder Decoder Multi-Head Attention

33



Why we used because this, These are to be used by each decoder in its 'Encoder-Decoder' attention layer and why we specifically



* Once I is comes (in previous o/p). Whatever o/p is generated in $t-1$ it will pass again as a previous o/p and it generated the one by one o/p!

429. Final Decoder Linear Softmax Layer!

34

which word in our vocab
is associated with this
index

am

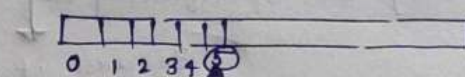
Get the index of the cell
with the Highest value
(argmax)

5

yaha jiski probability
High Ho! wo o/p
ayenga!

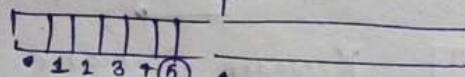
Log-probs

Logits



Softmax

Use for multiclass
classification!



Linear

corresponding to
score of unique
words!

Decoder stack o/p!



Linear :- Is a simple fully connected neural networks that projects
the vector produced by the Stack of Decoder → Logits

vector

Logits_vector :- model → 10000 → Logits vector
vocabulary of (10000 words)
unique words cells wide

- (2) Softmax Layer Turns those score into probabilities (add all upto 1.0)
The cell with the Highest probability is chosen and
the words associated with it is produced as the o/p. → Time Stamp!

Talking About the Recap of Training!

words	a	am	i	thanks	Student	<eos>
index	0	1	2	3	4	5

OHE

if i want to
Converted

Merci → Thanks

with an untrained
model!

Now using Embedding Technique (OHE)

0	1.0	0	0	0	0
---	-----	---	---	---	---

for the word 'am'

Untrained model o/p

0.2	0.2	0.1	0.2	0.2	0.1
-----	-----	-----	-----	-----	-----

Here we Basically we need to
calculate the loss functionImp for Back
propagation
will specifically
Happen!

Correct and desired o/p

0.0	0.0	0.0	1.0	0.0	0.0
-----	-----	-----	-----	-----	-----

a am i thanks Student <eos>

we need this value as
1.0So my Target model something
like this :-

o/p vocabulary :- a am i thanks student <eos>

posi ⁿ 1	0.0	0.0	1.0	0.0	0.0	0.0
posi ⁿ 2	0.0	1.0	0.0	0.0	0.0	0.0
posi ⁿ 3	1.0	0.0	0.0	0.0	0.0	0.0
posi ⁿ 4	0.0	0.0	0.0	0.0	1.0	0.0
posi ⁿ 5	0.0	0.0	0.0	0.0	0.0	1.0

(Before Training)

i/p

a am i
Student

o/p

I am a
StudentEach words
convert into
OHE!

* Trained model o/p

o/p vocab	a	am	i	thank	student	<eos>
posi ⁿ 1	0.01	0.02	0.93	0.01	0.03	0.01
posi ⁿ 2	0.01	0.8	0.1	0.05	0.01	0.03
posi ⁿ 3	0.09	0.001	0.001	0.001	0.002	0.001
posi ⁿ 4	0.001	0.002	0.001	0.02	0.94	0.01
posi ⁿ 5	0.1	0.01	0.001	0.001	0.001	0.98

• This is because of
loss function and Epochsie Keeps on updating